

PRACTICAL EXAMPLES EXECUTIONS

2520030395

MONISH.P

1) Virtual memory

```
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

```
monish_putti@monhh: ~/OSSP_Practical
monish_putti@monhh:~$ cd ~/OSSP_Practical
monish_putti@monhh:~/OSSP_Practical$ nano address.c
monish_putti@monhh:~/OSSP_Practical$ gcc address.c -o address
monish_putti@monhh:~/OSSP_Practical$ ./address
Value of x = 10
Virtual address of x = 0x7ffccb80e6f4
monish_putti@monhh:~/OSSP_Practical$
```

2) memory_layout

```
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {
    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address      : %p\n", (void *)main);
    printf("Data address       : %p\n", (void *)&global);
    printf("BSS address        : %p\n", (void *)&bss_variable);
    printf("Heap address       : %p\n", (void *)heap);
    printf("Stack address      : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

```
monish_putti@monhh: ~/OSSP_Practical
monish_putti@monhh:~$ cd ~/OSSP_Practical
monish_putti@monhh:~/OSSP_Practical$ nano memory_layout.c
monish_putti@monhh:~/OSSP_Practical$ gcc memory_layout.c -o memory_layout
monish_putti@monhh:~/OSSP_Practical$ ./memory_layout
Code address   : 0x5dfa3c1071a9
Data address   : 0x5dfa3c10a010
BSS address    : 0x5dfa3c10a018
Heap address   : 0x5dfa6d404010
Stack address  : 0x7fffb477276c
monish_putti@monhh:~/OSSP_Practical$
```

3) Demand Paging

```
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

```
monish_putti@monhh: ~/OSSP_Practical
monish_putti@monhh:~$ cd ~/OSSP_Practical
monish_putti@monhh:~/OSSP_Practical$ nano memory_layout.c
monish_putti@monhh:~/OSSP_Practical$ gcc memory_layout.c -o memory_layout
monish_putti@monhh:~/OSSP_Practical$ ./memory_layout
Code address   : 0x5dfa3c1071a9
Data address   : 0x5dfa3c10a010
BSS address    : 0x5dfa3c10a018
Heap address   : 0x5dfa6d404010
Stack address  : 0x7fffb477276c
monish_putti@monhh:~/OSSP_Practical$ cd ~/OSSP_Practical
monish_putti@monhh:~/OSSP_Practical$ nano page_table.c
monish_putti@monhh:~/OSSP_Practical$ gcc page_table.c -o page_table
monish_putti@monhh:~/OSSP_Practical$ ./page_table
Enter page number (0-3): 0
Page is already in memory.
Physical frame = 1
monish_putti@monhh:~/OSSP_Practical$
```

4) Address translation

```
#include <stdio.h>

int main() {
    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;
    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number = %d\n", page);
    printf("Offset = %d\n", offset);
    printf("Frame Number = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

```
monish_putti@monhh: ~/OSSP_Practical
Enter page number (0-3): 0
Page is already in memory.
Physical frame = 1
monish_putti@monhh:~/OSSP_Practical$ nano address_translation.c
monish_putti@monhh:~/OSSP_Practical$ gcc address_translation.c -o address_translation
monish_putti@monhh:~/OSSP_Practical$ ./address_translation
Virtual Address = 250
Page Number = 2
Offset = 50
Frame Number = 1
Physical Address = 150
monish_putti@monhh:~/OSSP_Practical$
```

5) Allocation memory

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");
    free(p);
    return 0;
}
```

```
monish_putti@monhh: ~/OSSP_Practical
Physical Address = 150
monish_putti@monhh:~/OSSP_Practical$ nano dynamic_memory.c
monish_putti@monhh:~/OSSP_Practical$ gcc dynamic_memory.c -o dynamic_memory
monish_putti@monhh:~/OSSP_Practical$ ./dynamic_memory
10 20 30 40 50
monish_putti@monhh:~/OSSP_Practical$
```

6) Accessing memory

```
#include <stdio.h>

int main() {
    int *p = NULL;
    printf("Before accessing memory\n");
    *p = 100;
    printf("After accessing memory\n");
    return 0;
}
```

```
monish_putti@monhh:~/OSSP_Practical$ nano accessing_memory.c
monish_putti@monhh:~/OSSP_Practical$ gcc null_pointer.c -o null_pointer
gcc1: fatal error: null_pointer.c: No such file or directory
compilation terminated.
monish_putti@monhh:~/OSSP_Practical$ gcc accessing_memory.c -o accessing_memory
monish_putti@monhh:~/OSSP_Practical$ ./accessing_memory
Before accessing memory
Segmentation fault
monish_putti@monhh:~/OSSP_Practical$
```